6CS012

Artificial Intelligence and Machine Learning

Task I: Question and Answer

Name	Swoyam Pokharel
Student ID	2431342
Group	15G
Tutor	Jinu Nyachhyon
Module Leader	Siman Giri

Abstract

This report covers Task I of the final portfolio assessment. The long answer focuses on production challenges in e-commerce machine learning systems. The first short answer discusses techniques used to reduce overfitting in deep learning. The second short answer compares convolutional neural networks and recurrent neural networks.

Contents

Long Question

Question

You are a Machine Learning Engineer at a rapidly growing e-commerce company responsible for deploying and maintaining ML systems in production. Identify and explain at least three real-world challenges encountered in ML systems. For each challenge, discuss consequences, practical solutions, trade-offs, cross-functional collaboration, and the role of modern tools.

Answer

In a real-world e-commerce ML system, deploying the model is a first step, arguably the bigger challenge is keeping the model reliable, fast, and useful in real-word production. ML in production, in the context of an e-commerce company, is difficult because features like user behaviour, products, prices, stock, campaigns, and fraud patterns can shift faster than the original training dataset.

For instance, a recommendation model trained on normal shopping behaviour may fail during a major sale, where users suddenly become more price-sensitive and buy different categories of products. This is data drift. If ignored, it can reduce conversion, show irrelevant products, or miss suspicious checkout behaviour. Uber's Michelangelo platform is a useful example here because Uber notes that production models may behave differently from offline test sets, so it monitors predictions, actual outcomes, feature distributions, and prediction distributions (Uber Michelangelo). For an e-commerce platform, the equivalent is monitoring click-through rate, add-to-cart rate, fraud alerts, and recommendation confidence over time. The trade-off is that outcome-based monitoring is more accurate but slower, while distribution monitoring is faster but less precise.

Another problem is data quality. An e-commerce ML system depends on product IDs, prices, ratings, inventory status, category labels, and user-event logs. If prices are missing, inventory status is outdated, or product categories are corrupted, the model may receive misleading inputs during inference. The model itself may be unchanged, but the predictions can still become wrong because the features no longer represent reality. For example, a recommendation model might promote out-of-stock products, or a search-ranking model might rank products under the wrong category. Amazon's Deequ is a direct example because it verifies large production datasets and stops bad datasets from being published downstream to ML pipelines (Amazon Deequ). The solution is data validation, anomaly detection, schema checks, and pipeline alerts before training and during inference. The trade-off is extra engineering overhead, but that overhead is cheaper than silently serving bad predictions.

A third challenge is latency and scalability. Product search, ad ranking, and checkout fraud detection sit directly inside the user's flow, so they cannot wait for a slow model. The problem is that these models often need many user, product, seller, and session features; fetching those features can become a bigger bottleneck than the model itself. DoorDash faced a similar problem: its ML systems needed billions of feature records and low-latency reads, so it optimized Redis-based feature serving using hashes, binary serialization, string hashing, and compression (DoorDash Feature Store). In e-commerce, the same idea maps to online feature stores, caching, batch prediction for non-urgent recommendations, model compression, and distributed serving. The trade-off is that faster systems require more infrastructure work, and sometimes a simpler model is better because the best offline model is useless if it slows down checkout or search.

A reliable production ML system is not bound to just the model or the pipeline. Data scientists can improve the model, but engineers still have to serve it reliably, and operations teams need

to provide feedback, as they see the business impact first, when systems like fraud detection, recommendations or search results fail. MLOps dashboards, retraining jobs, and LLM-based log analysis are useful support tools. They make failures easier to detect and debug, but they do not make the system safe by themselves.

Short Question 1: Overfitting

Question

Overfitting is a common challenge in deep learning, especially when models have high capacity. Describe at least two techniques used to reduce overfitting. For each technique, explain the intuition, how it affects training, and provide a real-world application example.

Answer

Overfitting happens when a model fits the training data too closely and starts learning noise, repeated phrases, or background details instead of the actual signal. One common fix is dropout. Dropout randomly disables some neurons during training, forcing the network to avoid relying on one narrow set of features. In Part III of this assessment, amongst the NLP experiments, adding dropout to the weighted LSTM improved accuracy from `0.7997` to `0.8114` and reduced errors from `993` to `935`, but macro F1 slightly decreased from `0.6815` to `0.6792`. In Part II, the vision task, the deeper CNN with dropout also took longer (`315.33s`) than the version without dropout (`270.61s`), while accuracy dropped from `1.0000` to `0.9983`. So the takeaway here is that dropout can make the model less fragile, but it can also make training longer, noisier, and does not guarantee improvement on every metric.

Early stopping is another practical technique. Early stopping is based on monitoring a chosen performance signal during training, usually a validation metric because validation data acts as a proxy for unseen data. The idea is to stop once that signal stops improving, rather than training until the model fully minimizes training loss. The intuition is that training should stop at the point where the model is no longer improving on unseen data, not when it has fully minimized training loss. A deep model can keep becoming better at the training set while its validation performance gets worse, which is a classic overfitting pattern. This helps in two ways: it reduces unnecessary training time and avoids pushing the model further into overfitting. Although not a perfect fix, because stopping too early can cause underfitting, it is a practical trade-off.

Short Question 2: Neural Network Architecture

Question

Explain the fundamental differences between a Convolutional Neural Network and a Recurrent Neural Network in terms of input structure, information flow, architecture design, and parameter sharing. Provide examples and briefly discuss training challenges such as vanishing/exploding gradients and overfitting.

Answer

A CNN and an RNN are built for different kinds of structure. A CNN is mainly used for grid-like data such as images, where nearby pixels matter. It treats the input as a spatial layout, so the position of pixels relative to each other is important. A convolution filter scans local regions and learns patterns such as edges, shapes, textures, and object parts. Pooling layers are often added to reduce spatial size while keeping the strongest features. This is why CNNs are useful for image-related tasks or any task where visual structure matters.

An RNN, on the other hand, is built for sequential data, where order matters. Instead of looking at the whole input, it processes one step at a time and updates a hidden state. That hidden state carries information from earlier steps, which makes RNNs useful for text classification, sentiment analysis, speech, and time-series prediction, which are cases where the order of input matters significantly.

Furthermore, they differ in terms of information flow and parameter sharing. In a CNN, information mostly flows from local feature extraction to higher-level features, then into a classifier. The same filter is reused across spatial locations, so the model can detect the same visual pattern whether it appears at the top, bottom, or middle of an image. This gives CNNs the ability to "translate knowledge" without needing separate parameters for every pixel position. RNNs also share parameters, but in a different way. They reuse recurrent weights across time steps so the model can repeatedly update a hidden state as it moves through a sequence. The key difference is that CNN's parameter sharing is spatial, while RNN sharing is temporal and tied to memory through the hidden state.

With that said, both architectures face training issues. RNNs especially suffer from vanishing or exploding gradients when sequences are long because gradients must flow through many time steps. LSTM/GRU layers help by controlling what information is remembered or forgotten, while gradient clipping prevents unstable updates. CNNs can also face gradient issues in very deep networks, although batch normalization and residual connections often help. Finally, both CNNs and RNNs can overfit when the model is too large for the dataset. Dropout, early stopping, regularization, and data augmentation are common fixes, with the right choice depending on the task.